

CSA1711 - Artificial Intelligence

N. Prince Raj - 192312284

Constraint-Based Problem Solving

CSP Backtracking · Heuristic Grid Search (A) · Online Search for a Rescue Robot*

Problem 1 — Doctor Shift Scheduling (CSP + Backtracking Search)

1.1 Problem Formulation

This is a Constraint Satisfaction Problem (CSP) defined as:

- Variables: D1, D2, D3 (the three doctors)
- Domain of each variable: {Morning, Afternoon, Night}
- Constraints:
 - (i) $D1 \neq \text{Night}$
 - (ii) $\text{shift}(D2) < \text{shift}(D3)$, using order Morning(1) < Afternoon(2) < Night(3)
 - (iii) All-Different(D1, D2, D3) — one doctor per shift
 - (iv) $D3 \neq \text{Morning}$

1.2 Domain Reduction (Node Consistency / Pre-processing)

Before search begins, unary constraints (i) and (iv) prune the domains directly:

- $D1 \in \{\text{Morning, Afternoon}\}$ (Night removed by constraint i)
- $D2 \in \{\text{Morning, Afternoon, Night}\}$ (no unary constraint)
- $D3 \in \{\text{Afternoon, Night}\}$ (Morning removed by constraint iv)

1.3 Backtracking Search Trace

Variable ordering used: $D1 \rightarrow D2 \rightarrow D3$. At every assignment, the constraints applicable so far are checked immediately (constraint propagation is applied incrementally).

Step	Assignment Attempted	Constraint Checked	Result
1	D1 = Morning	(i) $D1 \neq \text{Night} \rightarrow$ satisfied	Consistent
2	D2 = Afternoon	(iii) All-different (M taken by D1) $\rightarrow$ satisfied	Consistent
3	D3 = Night	(iv) $D3 \neq \text{Morning} \rightarrow$ satisfied; (iii) all-different $\rightarrow$ satisfied	Consistent
4	Check (ii)	$D2(\text{Afternoon})=2 < D3(\text{Night})=3 \rightarrow$ satisfied	Solution FOUND
5	Backtrack to D2 (explore alt.)	D2 = Night tried next	—
6	D3 domain after D1=M, D2=N	Only 'Afternoon' left; but (ii) needs $D2 < D3 \rightarrow N(3) < A(2)$ false	Failure – backtrack
7	Backtrack to D1, try D1 = Afternoon	(i) $D1 \neq \text{Night} \rightarrow$ satisfied	Consistent

Step	Assignment Attempted	Constraint Checked	Result
8	D2 = Morning	(iii) all-different → satisfied	Consistent
9	D3 = Night	(iv) D3 ≠ Morning satisfied; (ii) M(1) < N(3) satisfied	Second solution FOUND
10	D2 = Night (alt. under D1=Afternoon)	Remaining shift for D3 = Morning, but (iv) forbids D3=Morning	Failure – backtrack
11	Backtrack to D1, try D1 = Night	(i) D1 ≠ Night violated immediately	Pruned (domain excluded)

1.4 Search Tree Summary

The backtracking tree explored two full, constraint-satisfying leaves (solutions) and pruned three partial branches early because they violated constraint (ii), (iv), or (i):

- Branch D1=Morning, D2=Afternoon, D3=Night → VALID
- Branch D1=Morning, D2=Night, D3=Afternoon → PRUNED (violates ii)
- Branch D1=Afternoon, D2=Morning, D3=Night → VALID
- Branch D1=Afternoon, D2=Night, D3=Morning → PRUNED (violates iv)
- Branch D1=Night, ... → PRUNED immediately (violates i, domain empty)

1.5 Final Valid Schedule

Doctor	Assigned Shift
D1	Morning
D2	Afternoon
D3	Night

A second equally valid schedule also exists (found later in the same search):

Doctor	Assigned Shift
D1	Afternoon
D2	Morning
D3	Night

Problem 2 — Robot Path Planning in a 5×5 Grid (Heuristic Search)

2.1 Environment Setup

S = Start, G = Goal, # = Obstacle, . = Free cell. The grid used for this trace is:

	Col 0	Col 1	Col 2	Col 3	Col 4
Row 0	S	.	.	.	.
Row 1	.	#	#	.	.
Row 2	.	.	.	#	.
Row 3	.	#	.	#	.
Row 4	.	.	.	.	G

Start S = (0,0). Goal G = (4,4). Obstacles at (1,1), (1,2), (2,3), (3,1), (3,3).

2.2 Note on Algorithm Choice

Plain Breadth-First Search (BFS) expands nodes level-by-level and does not use a heuristic at all — it treats every frontier node as equally promising. Since the problem explicitly asks for the Manhattan-distance heuristic to guide expansion, the heuristic can only be used meaningfully within a best-first scheme. The trace below therefore performs an $f(n) = g(n) + h(n)$ best-first expansion (i.e., BFS generalized to A* search using the Manhattan heuristic), which reduces to ordinary BFS in structure (queue-based, layer-like expansion) but uses $f(n)$ to order the priority queue instead of pure insertion order. This is the standard way "BFS + heuristic" is implemented for grid pathfinding.

2.3 Heuristic Function

$$h(n) = |r_{\text{goal}} - r_n| + |c_{\text{goal}} - c_n| = |4 - r| + |4 - c| \quad (\text{Manhattan Distance to Goal (4,4)})$$

This heuristic is admissible (never overestimates the true remaining cost) and consistent, because each move changes row or column by exactly 1 and costs exactly 1.

2.4 Step-by-Step Node Expansion and Priority-Queue Updates

$g(n)$ = cost from Start to n. $f(n) = g(n) + h(n)$. Ties are broken by insertion order (first-in-first-served among equal f-values, mirroring BFS's FIFO behaviour).

#	Node Expanded (r,c)	$g(n)$	$h(n)=\text{Manhattan}$	$f(n)=g+h$	Frontier (Priority Queue) After Expansion
1	(0,0) S	0	8	8	(0,1):8, (1,0):8
2	(0,1)	1	7	8	(1,0):8, (0,2):8
3	(1,0)	1	7	8	(0,2):8, (2,0):8
4	(0,2)	2	6	8	(2,0):8, (0,3):8
5	(2,0)	2	6	8	(0,3):8, (3,0):8, (2,1):8
6	(0,3)	3	5	8	(3,0):8, (2,1):8, (0,4):8, (1,3):8
7	(3,0)	3	5	8	(2,1):8, (0,4):8, (1,3):8, (4,0):8
8	(2,1)	3	5	8	(0,4):8, (1,3):8, (4,0):8, (2,2):8

#	Node Expanded (r,c)	g(n)	h(n)=Manhattan	f(n)=g+h	Frontier (Priority Queue) After Expansion
9	(0,4)	4	4	8	(1,3):8, (4,0):8, (2,2):8, (1,4):8
10	(1,3)	4	4	8	(4,0):8, (2,2):8, (1,4):8
11	(4,0)	4	4	8	(2,2):8, (1,4):8, (4,1):8
12	(2,2)	4	4	8	(1,4):8, (4,1):8, (3,2):8
13	(1,4)	5	3	8	(4,1):8, (3,2):8, (2,4):8
14	(4,1)	5	3	8	(3,2):8, (2,4):8, (4,2):8
15	(3,2)	5	3	8	(2,4):8, (4,2):8
16	(2,4)	6	2	8	(4,2):8, (3,4):8
17	(4,2)	6	2	8	(3,4):8, (4,3):8
18	(3,4)	7	1	8	(4,3):8, (4,4)-GOAL:8
19	(4,3)	7	1	8	(4,4)-GOAL confirmed:8
20	(4,4) G — GOAL	8	0	8	Search terminates — goal popped from frontier

2.5 Optimal Path Found

Backtracking the parent pointers from the Goal gives the optimal path (one of two symmetric optimal paths — the trace above reaches the Goal via the bottom row):

- $(0,0) \rightarrow (1,0) \rightarrow (2,0) \rightarrow (3,0) \rightarrow (4,0) \rightarrow (4,1) \rightarrow (4,2) \rightarrow (4,3) \rightarrow (4,4)$

An equally optimal alternative path (top-then-right route) is:

- $(0,0) \rightarrow (0,1) \rightarrow (0,2) \rightarrow (0,3) \rightarrow (0,4) \rightarrow (1,4) \rightarrow (2,4) \rightarrow (3,4) \rightarrow (4,4)$

Both paths avoid every obstacle and consist of 8 moves.

2.6 Result

Metric	Value
Optimal path length	8 moves
Optimal path cost (1 per move)	8
f(n) at goal	8 (matches f at start $\Rightarrow$ heuristic was consistent throughout)
Nodes expanded before goal popped	19

Problem 3 — Autonomous Rescue Robot in a Partially Observable Environment

a) Analysis of Constraints and Their Effect on Decision-Making

Movement restricted to 4 directions

This discretizes the state space into a grid graph, keeping the branching factor small (≤ 4) so search remains tractable in real time — important for a rescue scenario where decisions must be fast.

Blocked cells (obstacles)

Obstacles remove edges from the state graph. Because the robot cannot see the whole map in advance, obstacles discovered mid-execution force it to revise its plan rather than follow a single pre-computed route — this is what pushes the problem toward an online search formulation rather than pure offline planning.

Limited sensing (only adjacent cells observable)

The environment is partially observable, so the robot cannot build a complete world model up front. It must interleave acting and sensing: move, observe the newly adjacent cells, update its internal map, and re-plan. This is the defining feature of an online search agent as opposed to an offline planner that computes a full solution before acting.

Uniform move cost (1) plus risky-zone penalty (+2)

This turns the problem into a weighted-graph shortest-path problem rather than a simple unweighted BFS problem. Cost is no longer proportional to number of steps; a longer but safer path can be cheaper than a shorter but riskier one, so the search must track and compare cumulative path cost, not just path length.

Minimizing total cost while ensuring safe navigation

This defines the robot's objective/performance measure: reach G at minimum cumulative cost, where cost is a proxy that already bundles in safety (risky zones cost more). It converts "be safe" from a separate constraint into a term the cost-minimizing search naturally respects.

Dynamic knowledge updates (online search)

Because the map is discovered incrementally, the robot must maintain a belief state (its current best map of the building) and be willing to discard/replan a previously chosen route whenever new information (a newly seen obstacle or risky zone) contradicts its assumptions.

b) Solution Approach: Online Search Agent using Uniform Cost Search (LRTA*-style)

Because the environment is partially observable and only locally sensed, a single offline Uniform Cost Search (UCS) over the full map is not directly possible — the robot does not have the full map. The recommended approach combines UCS as the local planner with an online search agent loop (in the style of Learning Real-Time A*, LRTA*):

- Step 1 — Initialize: Robot starts at S with an internal map containing only what it currently senses (its own cell + adjacent cells). All unseen cells are assumed free/unknown until observed.
- Step 2 — Local planning: Run Uniform Cost Search on the currently known map from the robot's position toward G, expanding the frontier in order of lowest cumulative path cost $g(n)$, where each edge costs 1 (normal) or 3 (1 move + 2 risk penalty, if the cell is known to be risky).
- Step 3 — Act: Execute the first move of the returned lowest-cost path.

- Step 4 — Sense: Observe the newly adjacent cells from the new position (obstacle? risky zone?) and update the internal map.
- Step 5 — Replan check: If the update revealed information that changes the cost/feasibility of the current plan (e.g., an assumed-free cell is actually an obstacle), discard the stale plan and re-run UCS from the current position (Step 2). Otherwise continue along the existing plan.
- Step 6 — Repeat Steps 2–5 until the robot's current cell equals G.
- Step 7 — Termination / failure handling: If UCS finds no path with the currently known map, the robot explores toward the nearest unknown region (frontier-based exploration) to gather more information rather than declaring failure prematurely.

Each time UCS is invoked, it behaves exactly like classical UCS: a priority queue ordered by cumulative cost $g(n)$, expanding the lowest-cost node first, guaranteeing an optimal path with respect to the currently known map. The "online" wrapper around it is what allows the robot to keep that guarantee meaningful as new information arrives, instead of committing irrevocably to a plan made with incomplete data.

c) AI Concepts Illustrated

Intelligent Agent

The robot is a goal-based (arguably utility-based) agent: it perceives its environment through local sensing (percepts = adjacent-cell status), maintains an internal state (its evolving partial map and current position), and selects actions (moves) via its search/planning component, forming a full perceive–think–act cycle.

Rationality

The agent is rational in the bounded sense: given what it currently knows (which is necessarily incomplete due to partial observability), it always selects the action that minimizes expected cumulative cost toward the goal while avoiding unsafe/blocked cells. It is not omniscient-rational (it cannot guarantee the globally optimal path from the very first step, since it doesn't know the whole map), but it is rational relative to its percept history — the standard definition of rationality for agents in partially observable environments.

Environment Type

Property	Classification for this Problem	Reason
Observability	Partially observable	Only adjacent cells are sensed
Determinism	Deterministic	Moves have certain, predictable outcomes
Episodic vs. Sequential	Sequential	Each move affects future options and cost
Static vs. Dynamic	Static (per given constraints)	The building layout does not change while the robot moves
Discrete vs. Continuous	Discrete	Grid-based positions, discrete moves and costs
Single vs. Multi-agent	Single-agent	Only the rescue robot is acting

d) Justification of the Chosen Approach

An online search agent driven by repeated Uniform Cost Search is the most suitable choice here for several converging reasons:

- Cost structure requires UCS, not plain BFS: because risky zones cost more (+2) than normal moves, the cheapest path is not necessarily the path with the fewest moves. Plain BFS (which minimizes step count) could select a shorter-but-riskier path that is actually more expensive; UCS explicitly minimizes cumulative $g(n)$, which is exactly the quantity the problem asks the robot to minimize.
- Partial observability rules out pure offline planning: A* or UCS run once over the full map assumes the map is already known. Since the robot can only sense adjacent cells, it physically cannot compute a single globally-optimal plan before moving — it must plan, act, sense, and replan, which is the online search paradigm.
- Robustness to newly discovered obstacles: The replanning loop (Steps 4–5) lets the robot gracefully update its route the moment a previously unknown obstacle or risky zone is discovered, rather than failing or blindly executing a now-invalid plan — critical in a disaster environment where the map may be uncertain.
- Optimality within available knowledge: At every replanning step, UCS still guarantees the best possible path given everything sensed so far, so the robot's behaviour degrades gracefully to "as good as currently knowable" rather than to an arbitrary heuristic guess.
- Practicality over pure A*: A* would need an admissible heuristic (e.g., Manhattan distance) and could offer faster convergence, and could be substituted step-for-step in place of UCS in Step 2 for efficiency; however, UCS is presented here as the baseline because it needs no domain-specific heuristic design and is guaranteed optimal purely from cost, which keeps the solution simple and robust to the added risky-zone cost structure.

In summary, the combination of an online search agent architecture (to handle partial observability and dynamic knowledge updates) with Uniform Cost Search as the repeated local planner (to correctly handle the non-uniform move costs from risky zones) directly satisfies every constraint stated in the problem, making it the most suitable approach.